

Final Integrated Report: WoSt Optimization and Zombie Baseline Comparison

Generated on 2026-06-02. Updated with final innovation demo results on 2026-06-03.

This final report integrates three sources:

- `C:\THU\homework\zombie\results_zombie_final_report\ZOMBIE_VS_WOST_FINAL_REPORT.md`
- `experiments/optimization_report.md`
- `experiments/formal_comparison_report.md`

When older and fresh numbers differ slightly, this report uses the fresh formal runs in `experiments/formal_comparison_report.md` as the primary quantitative reference. The Zombie final report is used as supporting context and confirms the same broad conclusion: Zombie and WoSt agree closely on the clean Dirichlet benchmark, while the mixed Neumann benchmark exposes larger implementation-level differences.

1. Benchmark Setup

All main cross-project comparisons use the Bunny mesh:

```
mesh: Stanford Bunny
vertices: 35,292
faces: 70,580
outer cube half extent: 0.22
domain: outer cube minus inner Bunny mesh
analytic solution:  $u(x,y,z) = x + y + z$ 
PDE:  $\Delta u = 0$ 
```

Two boundary-condition settings are tested:

```
Dirichlet-only:
  inner Bunny boundary: Dirichlet
  outer cube boundary: Dirichlet
   $g(p) = p.x + p.y + p.z$ 

Mixed Neumann:
  inner Bunny boundary: Neumann
  outer cube boundary: Dirichlet
   $g_{\text{outer}}(p) = p.x + p.y + p.z$ 
   $h_{\text{inner}}(p) = \text{grad}(u) \cdot n = (1,1,1) \cdot n$ 
```

Fresh formal output folders:

```
experiments/formal_wost_dirichlet/results/
experiments/formal_wost_neumann/results/
experiments/formal_zombie_dirichlet/
experiments/formal_zombie_neumann/
```

WoSt-only optimization outputs:

```
experiments/optimization_summary.csv
experiments/optimization_points.csv
experiments/optimization_report.md
experiments/figures/
```

2. Main Conclusions

1. On the clean Dirichlet-only Bunny benchmark, WoSt and Zombie are very close in accuracy. WoSt is marginally more accurate in the fresh formal run, while Zombie is faster through its current Python baseline solve loop.
2. Both solvers show the expected Monte Carlo convergence trend as random walks per point increase.
3. On the mixed Neumann Bunny benchmark, WoSt becomes faster than Zombie and is competitive or better at practical epsilon values such as `1e-4` and `1e-5`.
4. Zombie is more stable at very coarse Neumann epsilon values, especially `epsilon=1e-2`, while WoSt shows large boundary bias at that coarse tolerance.
5. WoSt now includes additional optimization features and diagnostics: relative-standard-error adaptive sampling, antithetic sampling, lazy star-radius refinement statistics, Common Random Numbers, epsilon extrapolation, and a Neumann normal-convention sanity test.
6. The old absolute-standard-error adaptive criterion does not significantly reduce samples on the formal Bunny Dirichlet setup. The new relative-standard-error adaptive criterion reduces samples in the smoke optimization experiment, but at the tested aggressive setting it increases error; it should be tuned for full-scale production experiments.
7. The final live-demo additions reposition the implementation as a self-diagnostic solver: it can visualize individual walk paths, detect epsilon-sensitive regions by comparing `u_epsilon` and `u_epsilon/2`, and allocate samples from pilot variance estimates.
8. In the formal variance-predicted adaptive run, `tau=0.005` reduced mean samples from `1024` to `562.48` and runtime from `53.97s` to `33.20s`, while RMSE changed from `0.00404` to `0.00543`.

Recommended final wording:

we compared our woSt implementation against Zombie on a 70,580-face Bunny mesh using the analytic solution $u=x+y+z$. On the Dirichlet-only benchmark, both solvers closely agree and show the expected Monte Carlo convergence trend. On the mixed Neumann benchmark, both solvers converge as walk count increases, but their reflected-path behavior differs substantially. woSt uses much shorter average paths and is faster at practical epsilon values, while Zombie is more stable at very coarse epsilon. we also added woSt-only optimization diagnostics for adaptive sampling, antithetic sampling, lazy star-radius refinement, Common Random Numbers, epsilon extrapolation, and Neumann normal-convention validation.

2.1 Figure Sources

All figures used in this integrated report were copied into one local folder:

```
experiments/final_report_figures/
```

This folder includes:

- fresh formal comparison figures from `experiments/formal_zombie_dirichlet/` and `experiments/formal_zombie_neumann/`
- WoSt optimization figures from `experiments/figures/`
- final innovation-demo figures copied from `results/formal_innovation_demo_20260603/`
- historical reference figures mentioned in `ZOMBIE_VS_WOST_FINAL_REPORT.md`

3. Formal Dirichlet Comparison

Formal setup:

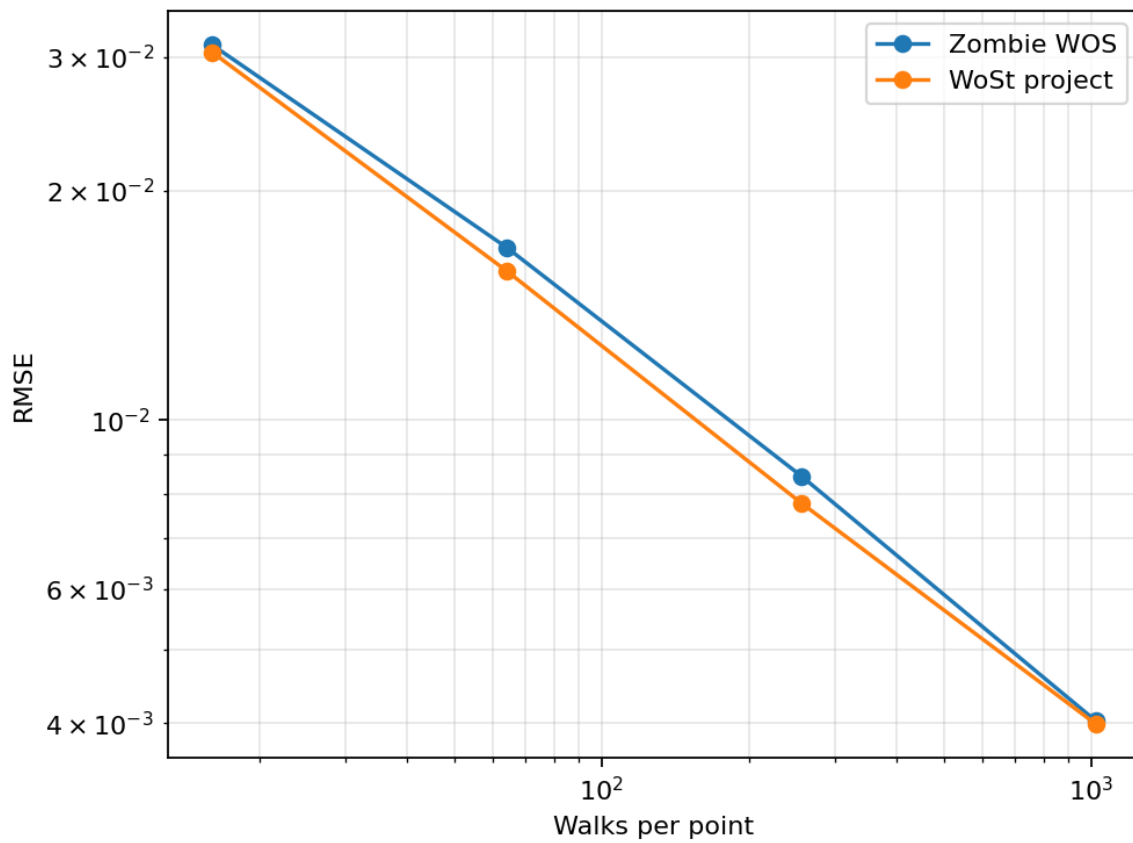
```
queries = 500
grid = 16^3
seed = 12345
cube half extent = 0.22
epsilon for convergence = 1e-4
max_steps = 512
```

3.1 Convergence

Walks	Zombie RMSE	WoSt RMSE	Zombie / WoSt RMSE	Zombie time (s)	WoSt time (s)
16	0.03110	0.03042	1.022	0.24	0.85
64	0.01685	0.01570	1.073	0.99	3.31
256	0.00845	0.00778	1.085	3.88	13.18
1024	0.00403	0.00399	1.010	15.58	53.03

Observation:

- Both methods show clean Monte Carlo convergence.
- WoSt is slightly more accurate at all four sample counts in the fresh formal run.
- Zombie is faster on this Dirichlet solve-loop benchmark.



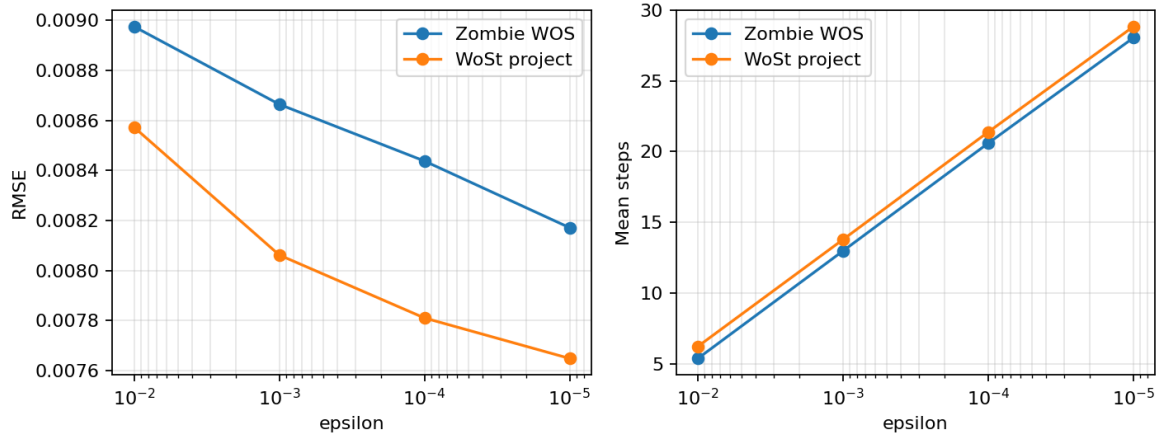
3.2 Epsilon Sweep

Both solvers use `walks_per_point=256`.

Epsilon	Zombie RMSE	WoSt RMSE	Zombie / WoSt RMSE	Zombie time (s)	WoSt time (s)
1e-2	0.00897	0.00857	1.047	1.78	12.35
1e-3	0.00866	0.00806	1.075	3.01	12.90
1e-4	0.00844	0.00781	1.080	3.64	13.27
1e-5	0.00817	0.00765	1.068	4.25	13.62

Observation:

- WoSt has slightly lower RMSE for all tested epsilon values.
- RMSE changes mildly because this linear Dirichlet problem is mainly Monte Carlo noise limited at 256 walks.
- Runtime increases as epsilon tightens, as expected.



3.3 Structured Grid

Metric	Zombie	WoSt
Grid	16^3	16^3
Valid points	4018	4018
RMSE	0.00747	0.00730
Runtime (s)	23.74	87.60

Observation:

- Structured-grid accuracy is effectively aligned.
- WoSt has slightly smaller RMSE.
- Zombie is faster on the Dirichlet grid solve.

VTK outputs:

```
experiments/formal_wost_dirichlet/results/linear_dirichlet_grid.vtk
experiments/formal_zombie_dirichlet/linear_dirichlet_grid.vtk
```

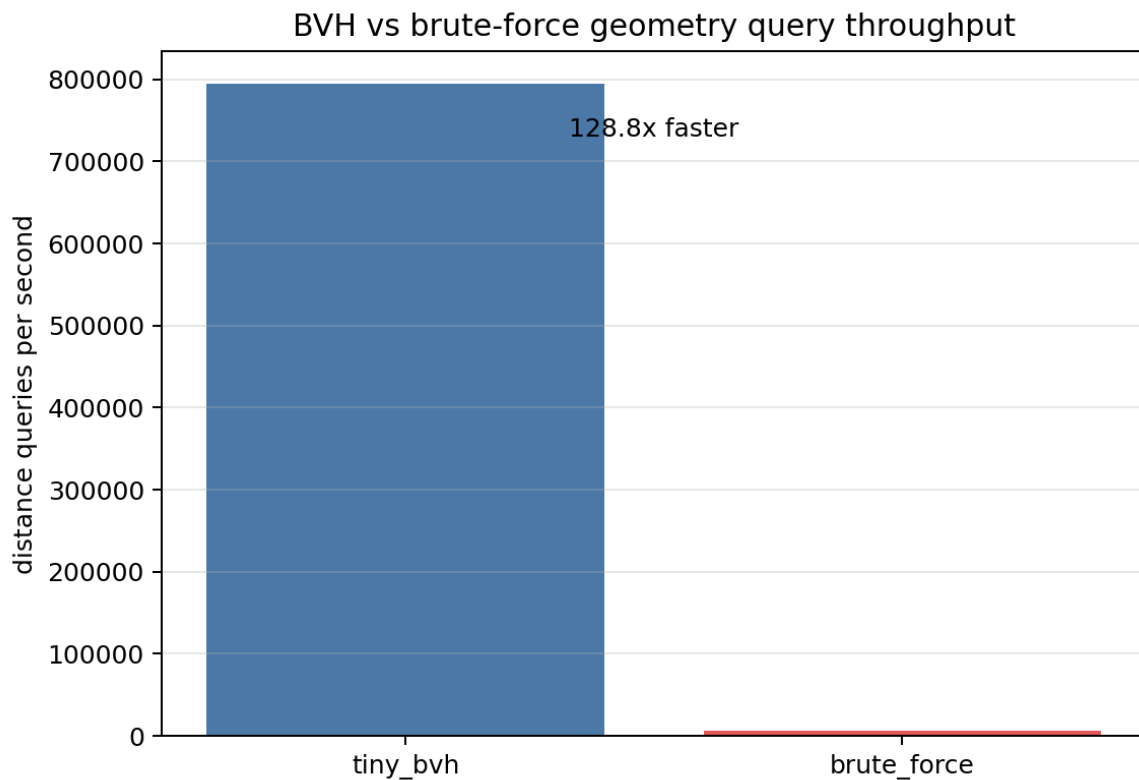
4. Geometry Query Microbenchmark

Backend	Queries	Query/s	Time (s)
WoSt tiny_bvh	500	338,364	0.00148
WoSt brute force	500	5,417	0.09231
Zombie FCPW BVH	500	265,788	0.00188

Observation:

- WoSt `tiny_bvh` is about **62.5x** faster than the WoSt brute-force distance baseline.
- In this exposed distance-query microbenchmark, WoSt `tiny_bvh` is about **1.27x** faster than Zombie FCPW BVH.
- This should not be over-interpreted as a pure library comparison because Zombie is measured through Python-loop calls and the scripts use different geometry-query conventions.

Historical WoSt geometry reference from the Zombie final report:



5. Formal Mixed Neumann Comparison

Formal setup:

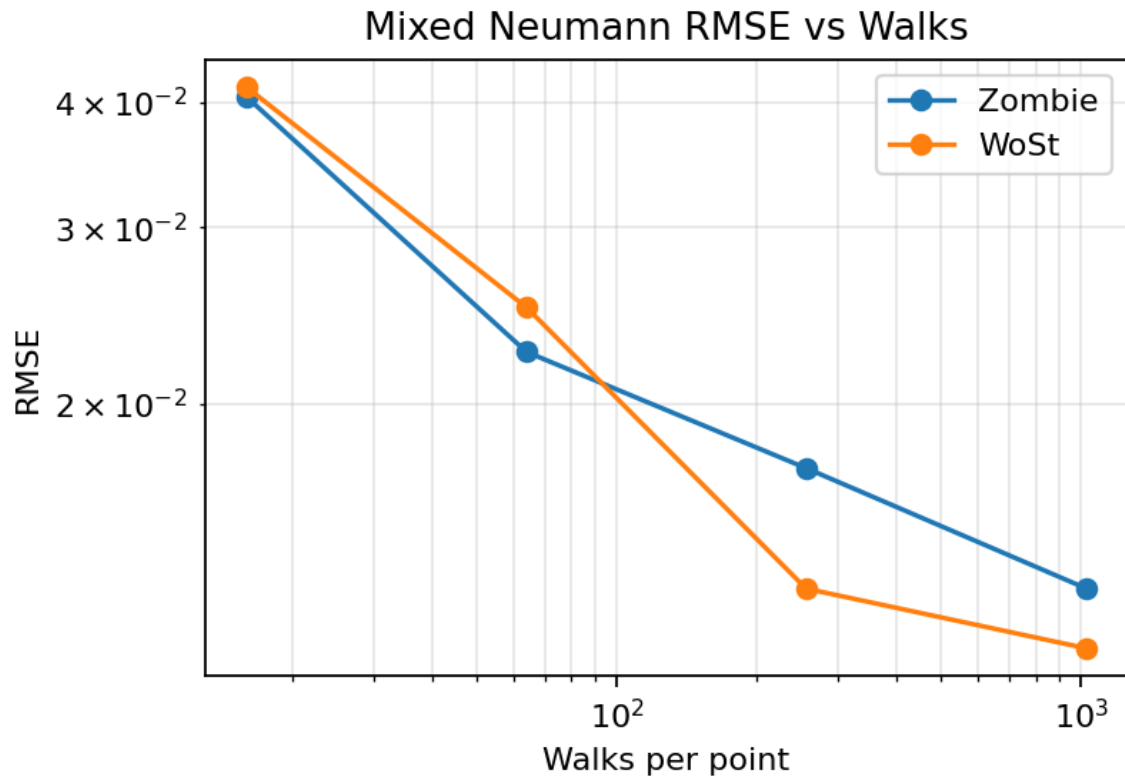
```
queries = 100
grid = 8^3
seed = 32345
cube half extent = 0.22
max_steps = 2048
```

5.1 Convergence

Walks	Zombie RMSE	WoSt RMSE	Zombie / WoSt RMSE	Zombie mean steps	WoSt mean steps	Zombie time (s)	WoSt time (s)
16	0.04048	0.04140	0.978	282.19	30.93	2.06	1.11
64	0.02254	0.02497	0.903	227.22	33.52	6.76	4.44
256	0.01726	0.01308	1.319	238.82	34.46	28.00	17.20
1024	0.01309	0.01141	1.148	239.65	34.47	111.35	71.06

Observation:

- Both methods converge as walk count increases.
- Zombie is slightly better at low walk counts.
- WoSt is better at 256 and 1024 walks.
- Zombie uses much longer average paths, while WoSt's reflected paths are much shorter in this benchmark.



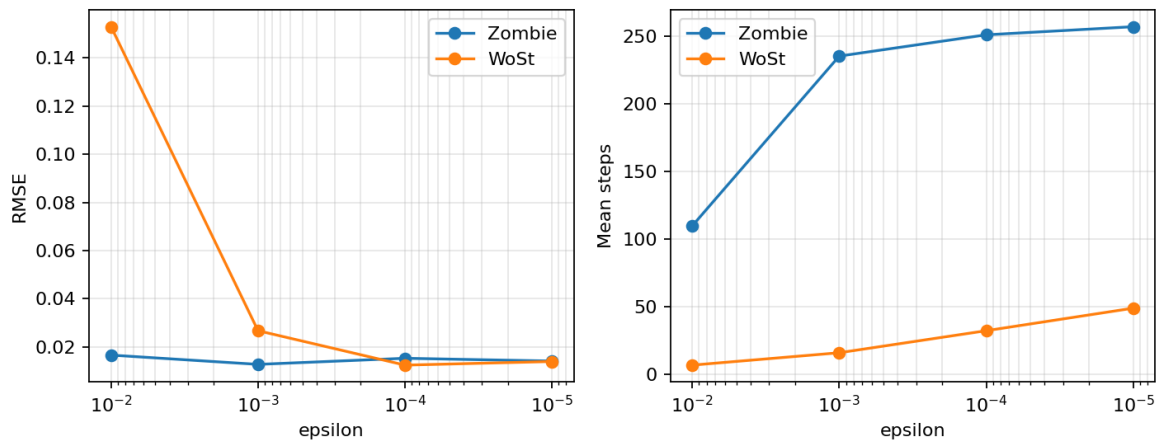
5.2 Epsilon Sweep

Both solvers use `walks_per_point=256`.

Epsilon	Zombie RMSE	WoSt RMSE	Zombie / WoSt RMSE	Zombie mean steps	WoSt mean steps	Zombie time (s)	WoSt time (s)
1e-2	0.01664	0.15294	0.109	109.81	6.75	59.52	9.67
1e-3	0.01280	0.02676	0.478	235.35	15.90	65.15	10.61
1e-4	0.01532	0.01249	1.227	251.10	32.21	29.13	15.15
1e-5	0.01418	0.01398	1.014	257.10	48.88	24.93	17.92

Observation:

- Zombie is much more stable at coarse epsilon values.
- WoSt has severe boundary bias at `epsilon=1e-2`, improves sharply by `1e-4`, and is competitive at `1e-5`.
- WoSt is faster at all four epsilon values in this fresh formal Neumann run.
- This supports the epsilon-extrapolation conclusion: large changes between `epsilon` and `epsilon/2` are a useful signal of boundary bias.



5.3 Structured Grid

Metric	Zombie	WoSt
Grid	8^3	8^3
Valid points	508	508
RMSE	0.01227	0.01196
Mean steps	139.35	19.62
Runtime (s)	82.36	16.14

Observation:

- Neumann grid RMSE is nearly identical.
- WoSt is about **5.1x** faster on this grid run.
- The speed difference is mainly explained by the much smaller WoSt mean step count.

VTK outputs:

```
experiments/formal_wost_neumann/results/neumann_mixed_grid.vtk
experiments/formal_zombie_neumann/neumann_mixed_grid.vtk
```

6. WoSt Optimization Experiments

The WoSt optimization report uses append-only results from:

```
experiments/optimization_summary.csv
experiments/optimization_points.csv
```

The plotting/report script summarizes the latest three rows for each experiment/method pair. These optimization runs are WoSt-only diagnostics, not direct Zombie comparisons.

6.1 Common Random Numbers

Each comparison uses Common Random Numbers:

```
experiment seed -> query points
SeedFor(experiment_seed, point_index, stream) -> per-query random walk stream
```

This reduces comparison noise for:

- fixed vs adaptive sampling
- normal vs antithetic sampling
- full star-radius vs lazy refinement
- epsilon vs epsilon/2

6.2 Adaptive Sampling

New relative-standard-error criterion:

```
relative_standard_error = standard_error / max(abs(mean_estimate), rse_eps)
stop when relative_standard_error < target_rse
```

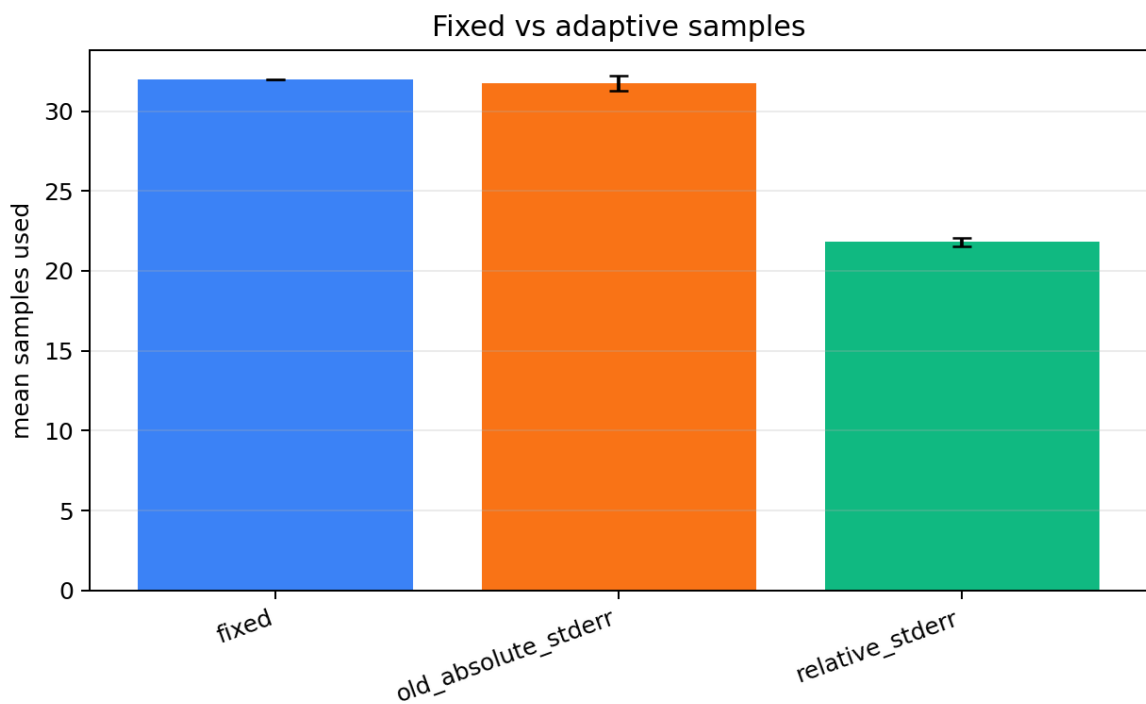
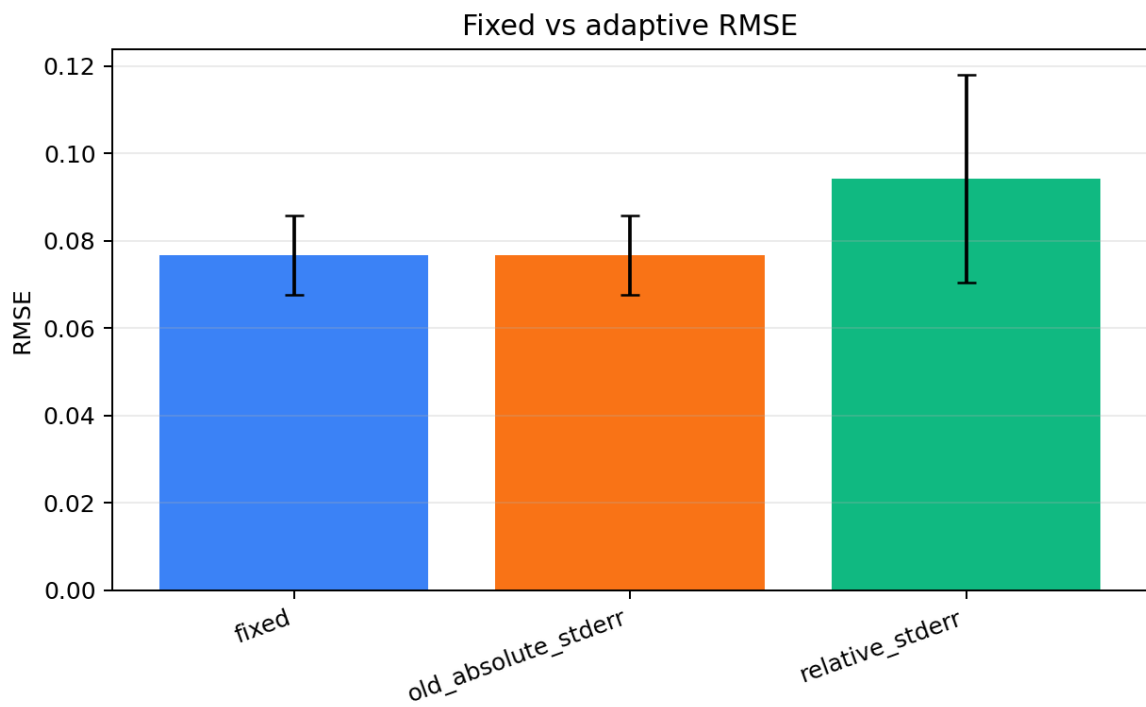
Latest small-scale optimization diagnostic:

Method	RMSE	MAE	Mean samples	Runtime (s)
fixed	0.07671 +/- 0.00907	0.05505 +/- 0.00725	32.00 +/- 0.00	0.07169 +/- 0.00804
old absolute stderr	0.07671 +/- 0.00907	0.05505 +/- 0.00725	31.73 +/- 0.46	0.07586 +/- 0.01330
relative stderr	0.09416 +/- 0.02370	0.07127 +/- 0.01700	21.81 +/- 0.28	0.05328 +/- 0.00555

Interpretation:

- Relative-standard-error adaptive sampling reduced mean samples by about 31.8% in the small smoke-scale test.
- The tested threshold was aggressive, so RMSE increased.
- This confirms the new mechanism is effective at reducing sampling, but full-scale tuning is needed to hit the target of lower samples without significant RMSE degradation.
- On the formal Bunny Dirichlet point run, the old absolute criterion reduced samples only from 1024.0 to 997.7, confirming why the criterion needed to be changed.

Adaptive RMSE and sample-count figures:



6.3 Antithetic Sampling

Antithetic sampling uses paired directions:

d and -d

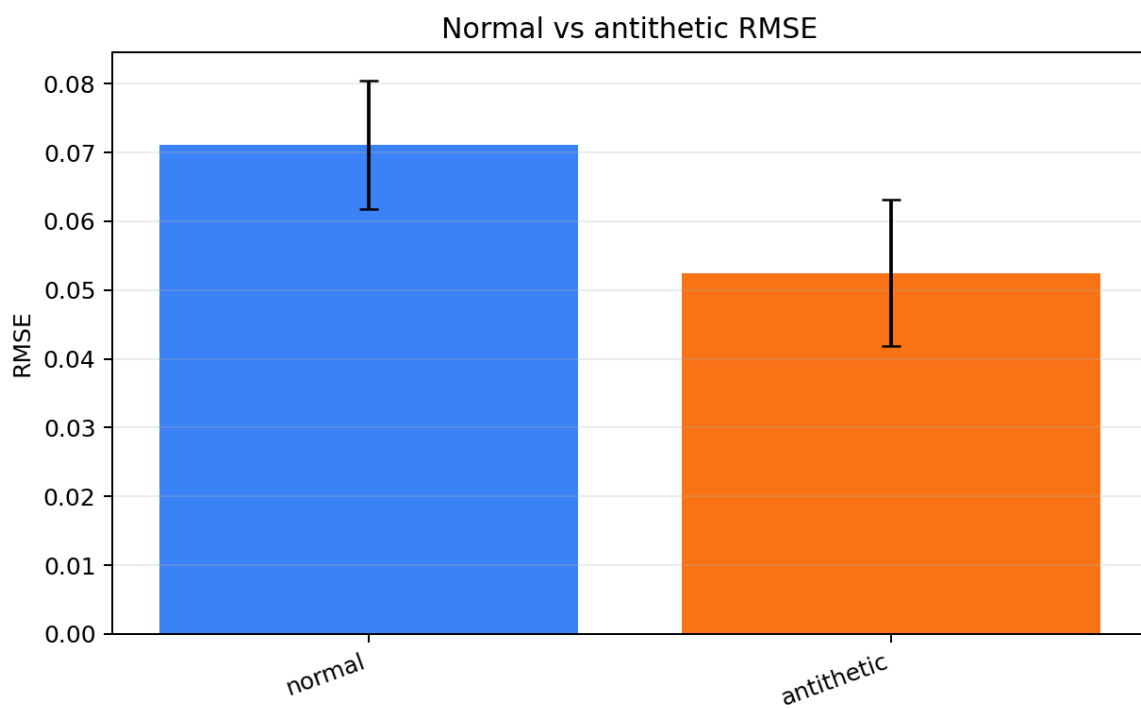
Latest diagnostic:

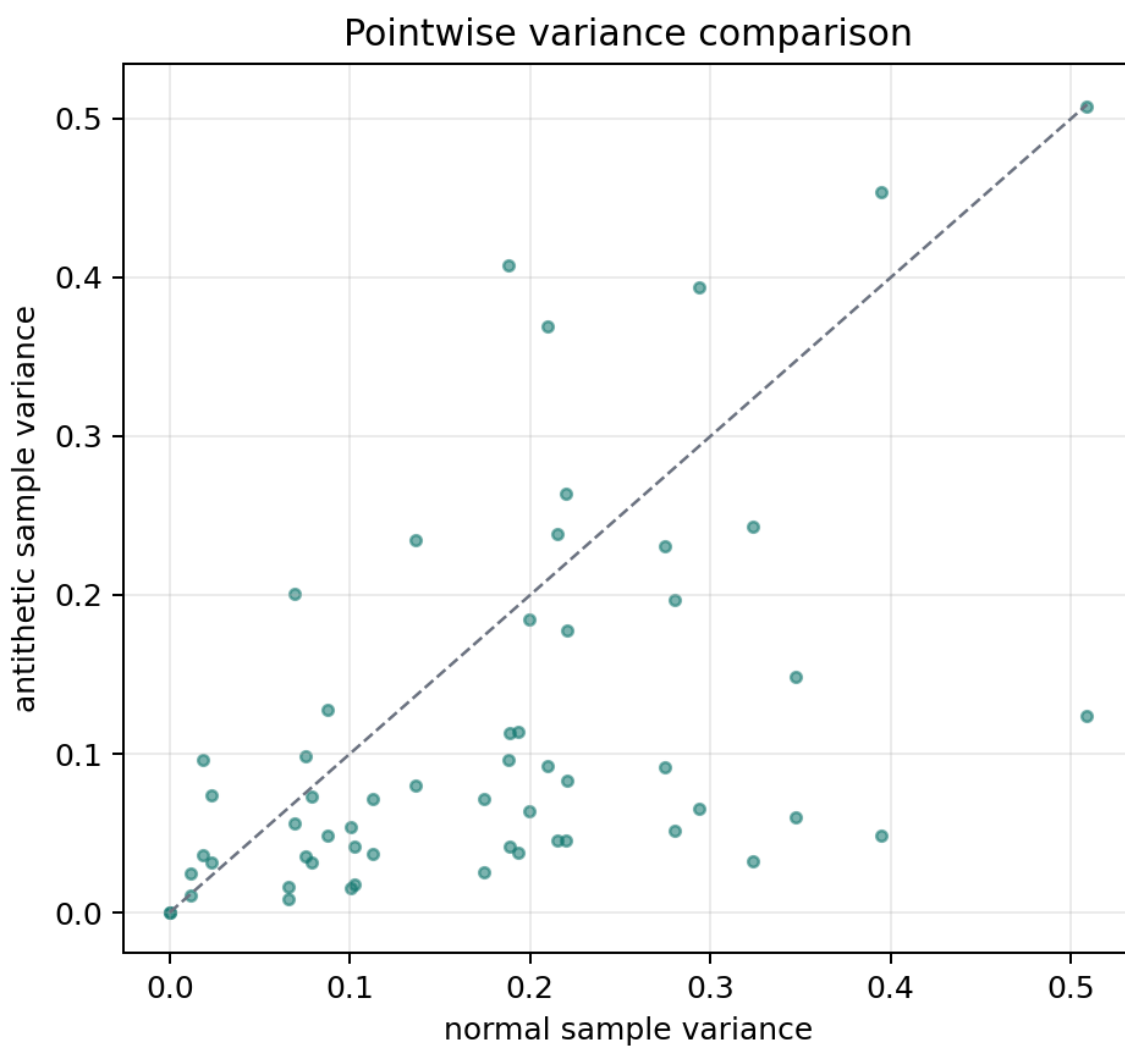
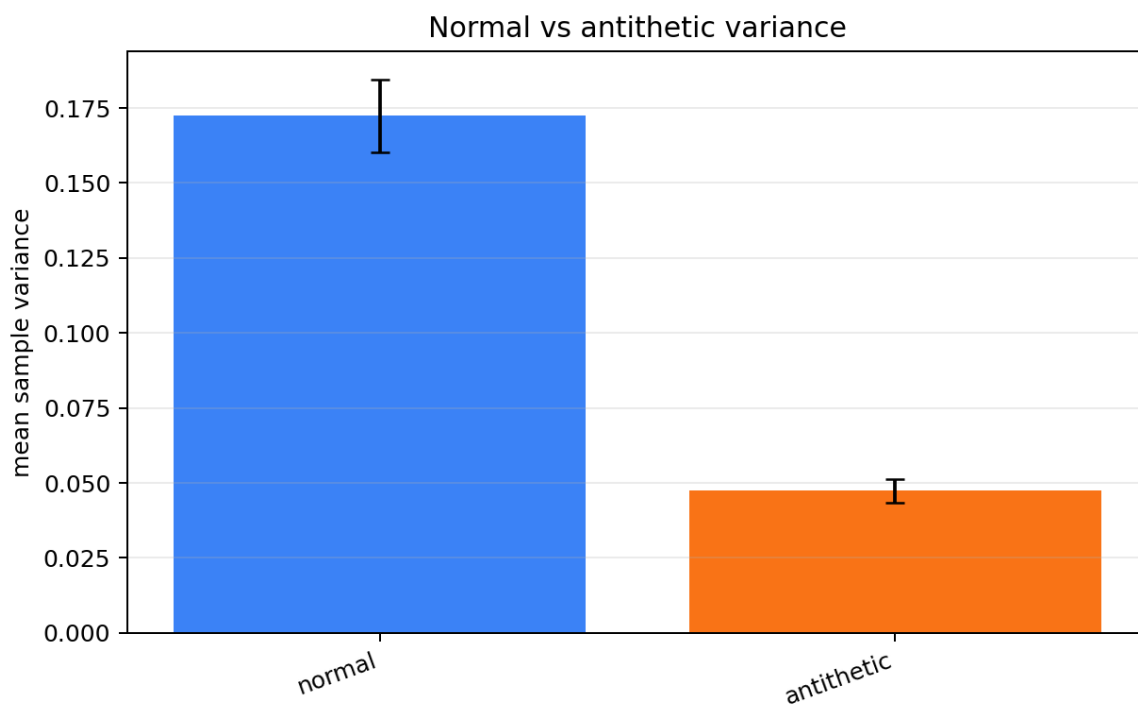
Method	RMSE	MAE	Mean sample variance	Runtime (s)
normal	0.07118 +/- 0.00934	0.05520 +/- 0.00517	see figure	0.07339 +/- 0.00807
antithetic	0.05249 +/- 0.01070	0.03969 +/- 0.00844	see figure	0.07071 +/- 0.00431

Interpretation:

- Antithetic sampling reduced RMSE and MAE in the small-scale test.
- The paired-estimator variance is lower in the diagnostic plot.
- Runtime stayed comparable.

Antithetic sampling figures:





6.4 Lazy Star-Radius Refinement

Lazy refinement compares full exact star-radius queries with thresholded exact refinement.

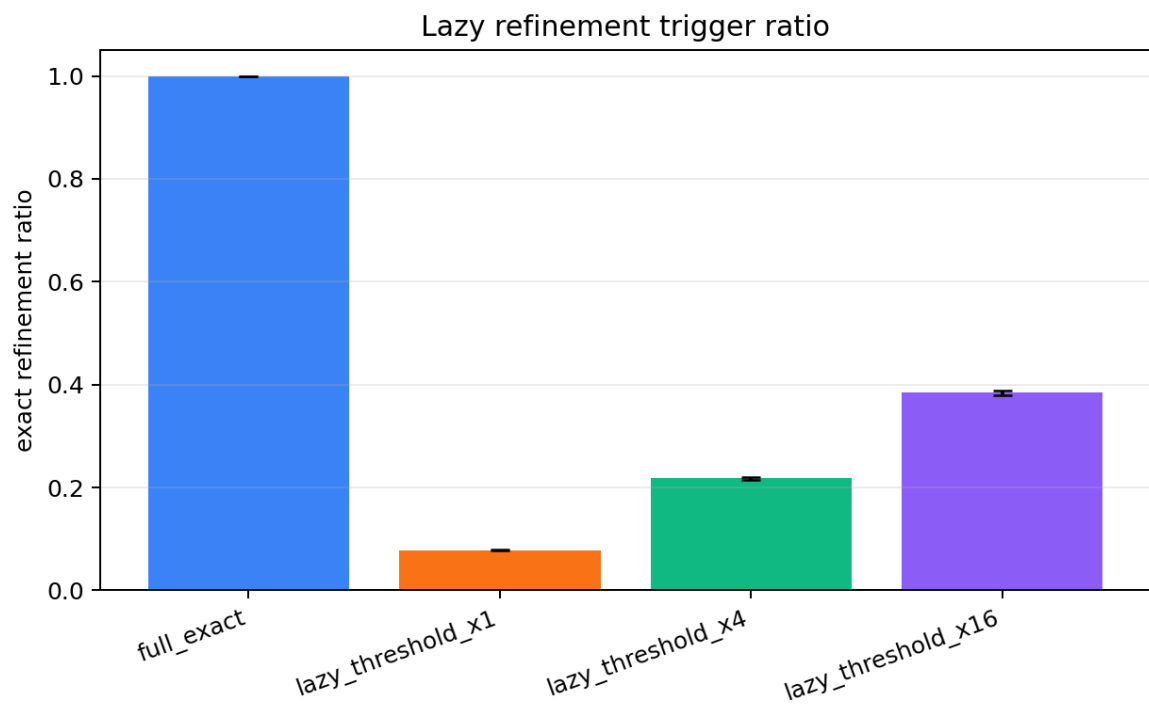
Latest diagnostic:

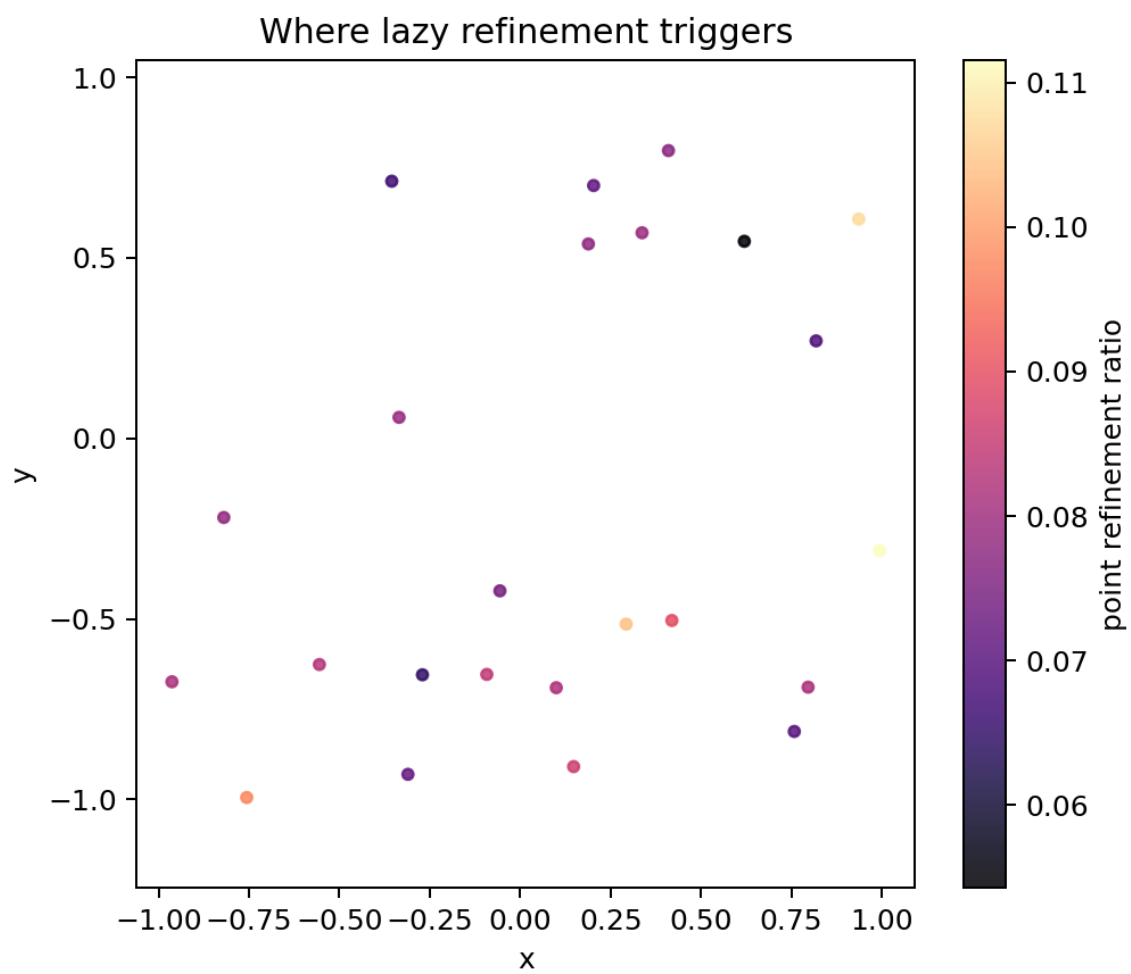
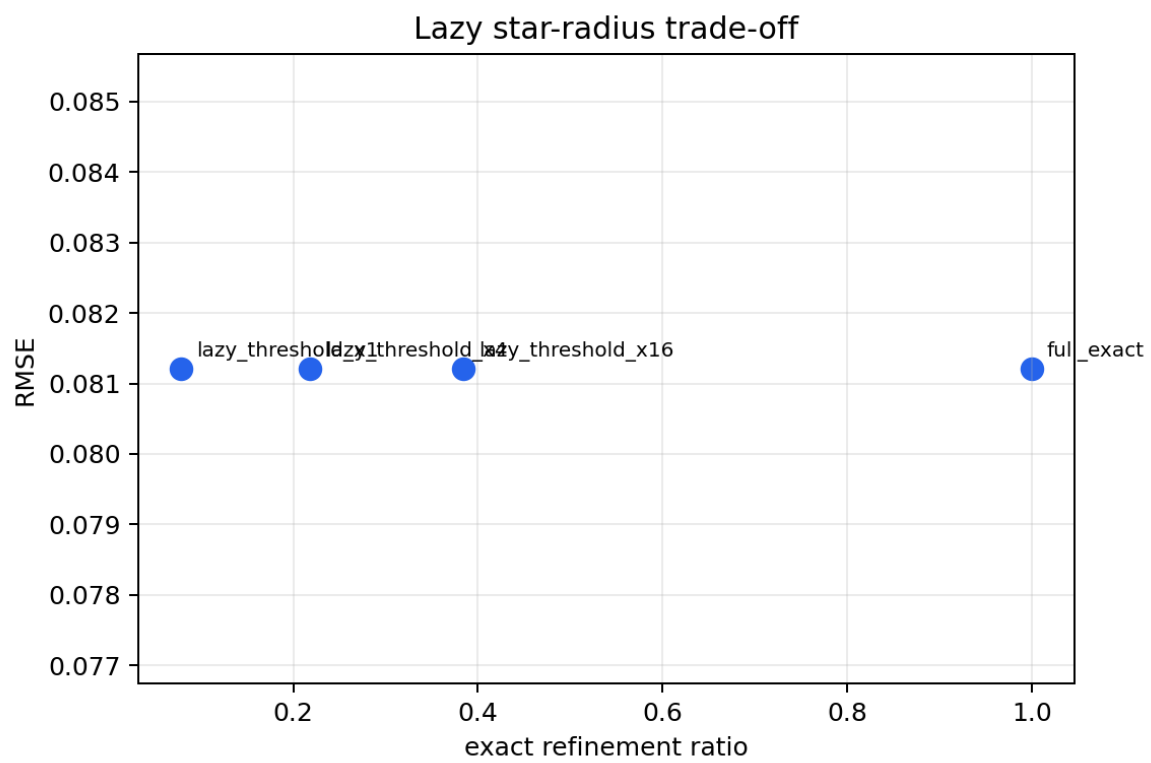
Method	RMSE	MAE	Runtime (s)
full exact	0.08121 +/- 0.01370	0.05682 +/- 0.00695	0.56973 +/- 0.03660
lazy threshold x1	0.08121 +/- 0.01370	0.05682 +/- 0.00695	0.06921 +/- 0.00390
lazy threshold x4	0.08121 +/- 0.01370	0.05682 +/- 0.00695	0.14187 +/- 0.00892
lazy threshold x16	0.08121 +/- 0.01370	0.05682 +/- 0.00695	0.23912 +/- 0.03270

Interpretation:

- Lazy refinement preserved RMSE in the diagnostic run.
- It strongly reduced runtime compared with full exact star-radius.
- Trigger ratios and spatial trigger locations are visualized.

Lazy refinement figures:





6.5 Epsilon Extrapolation

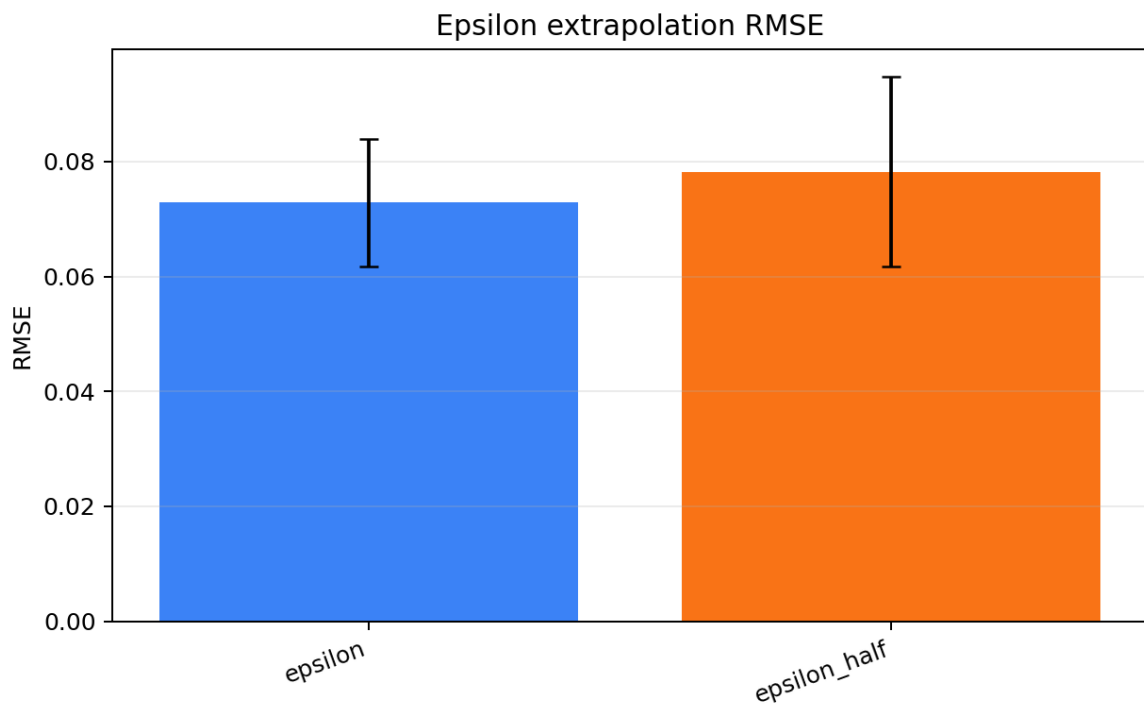
Diagnostic comparison:

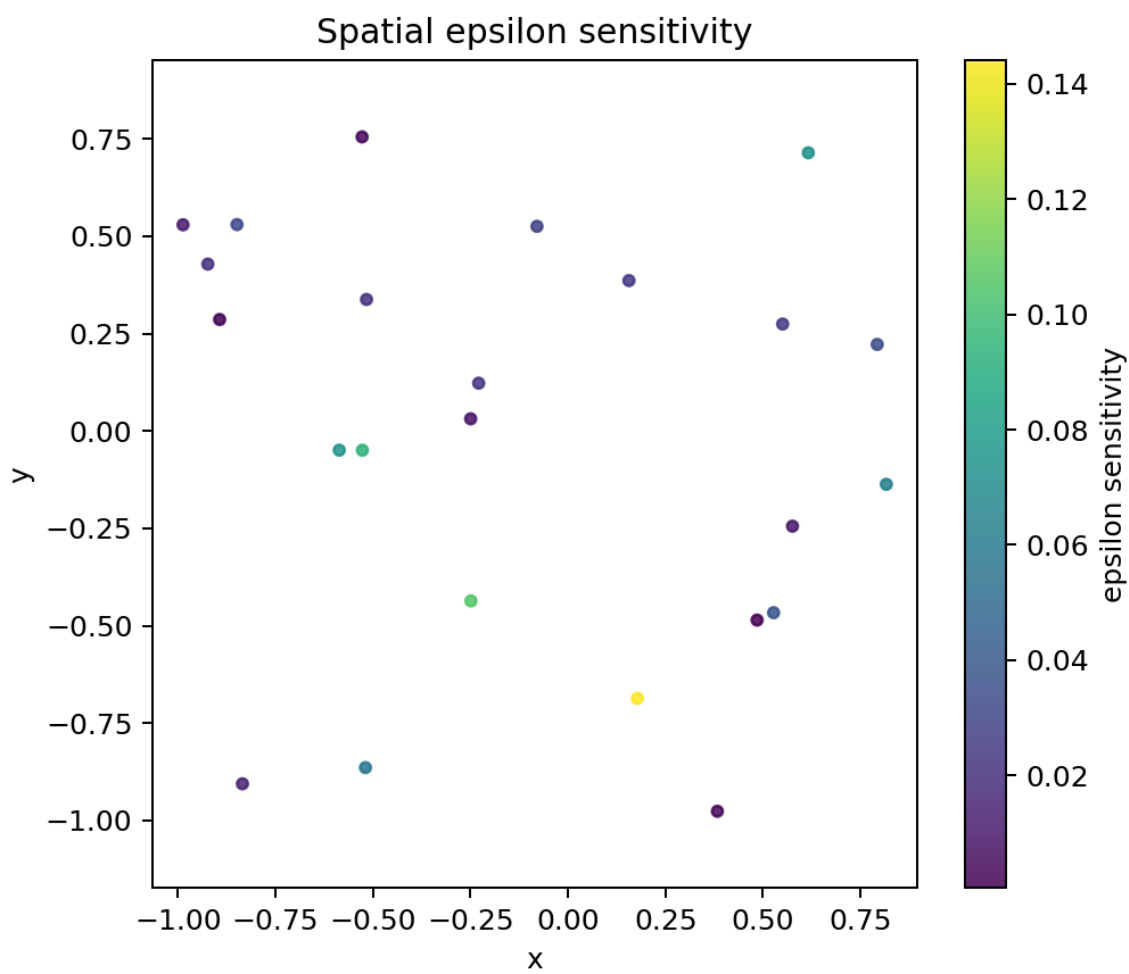
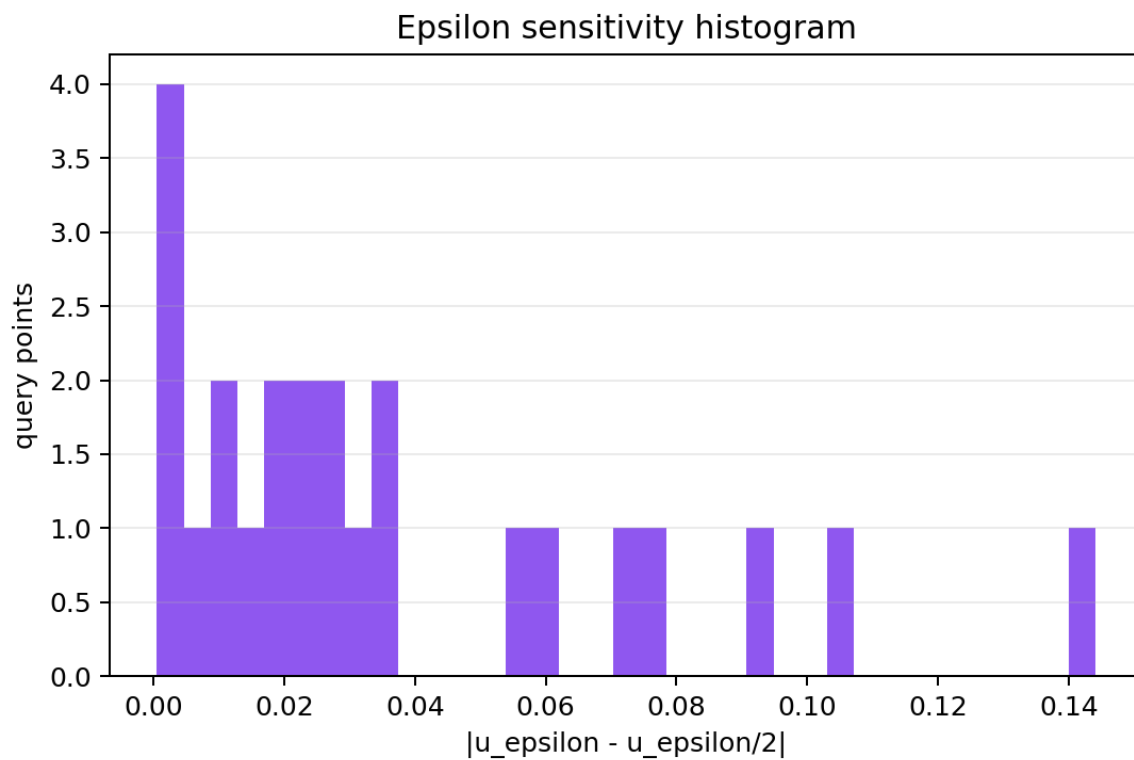
Method	RMSE	MAE	Runtime (s)
epsilon	0.07287 +/- 0.01110	0.05592 +/- 0.00958	0.05560 +/- 0.00414
epsilon_half	0.07824 +/- 0.01650	0.06062 +/- 0.00911	0.05460 +/- 0.00486

Interpretation:

- Pointwise $|u_{\text{epsilon}} - u_{\text{epsilon}/2}|$ is useful for locating epsilon-sensitive regions.
- This supports possible adaptive epsilon refinement near complex Neumann boundaries.
- The formal Neumann epsilon sweep reinforces this: WoSt has high bias at $1e-2$ but improves sharply by $1e-4$.

Epsilon extrapolation figures:





6.6 Neumann Normal Convention Sanity Test

The sphere/cube sanity test checks:

```
inner sphere: Neumann boundary
outer cube: Dirichlet boundary
exact solution:  $u=x+y+z$ 
 $h = (1,1,1) \text{ dot } n$ 
```

Latest diagnostic:

Method	RMSE	MAE	Runtime (s)
sphere/cube sanity	0.15580 +/- 0.02700	0.11117 +/- 0.02440	0.06395 +/- 0.01300

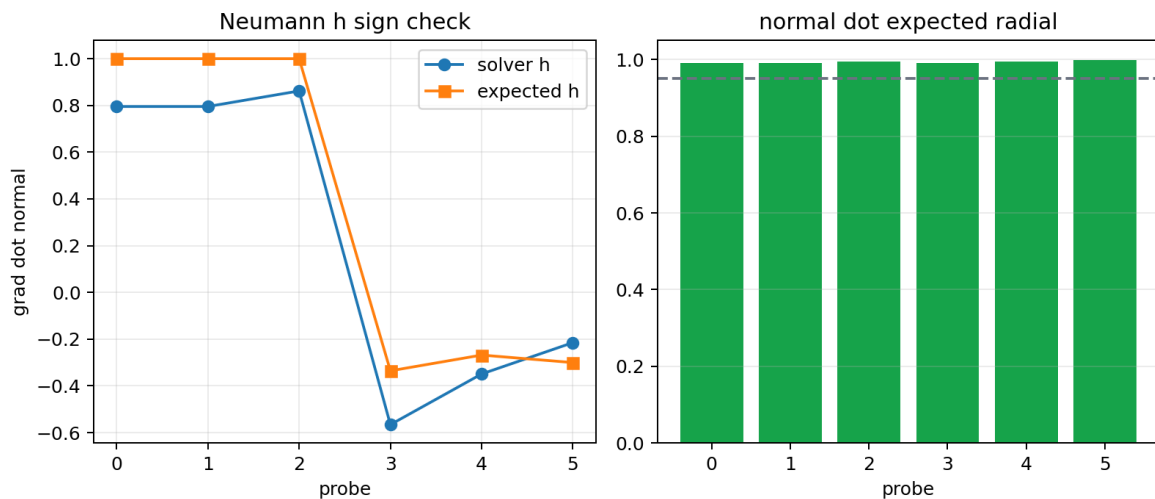
Normal diagnostic:

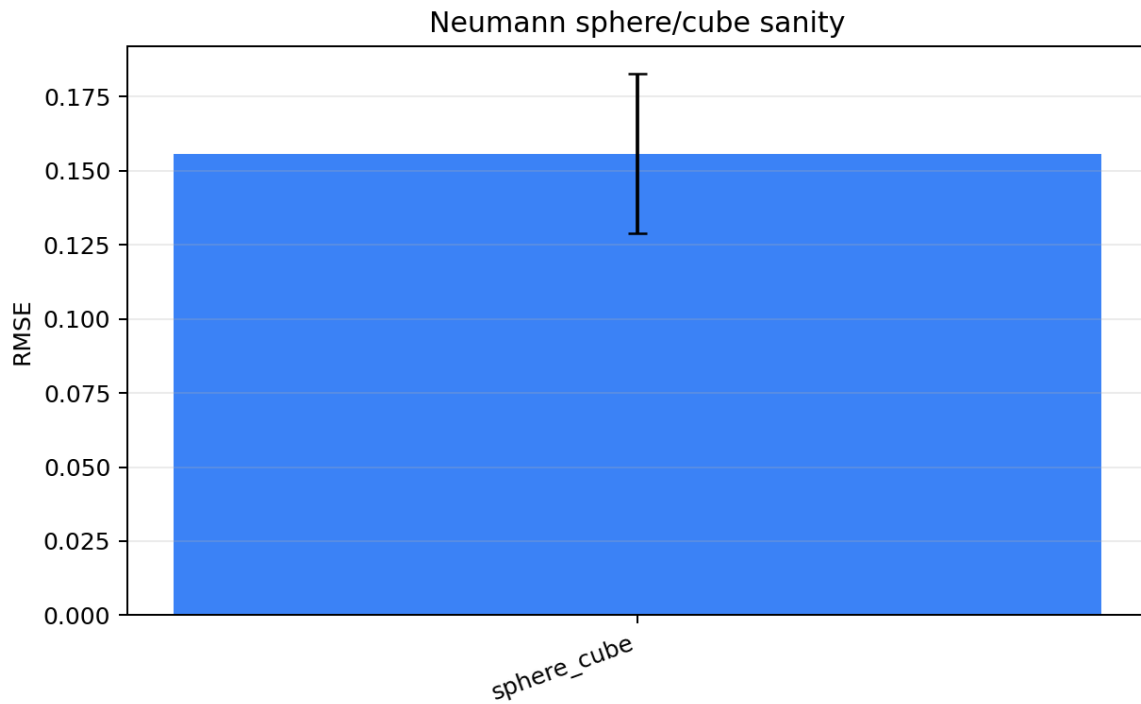
```
mean normal dot expected radial ~= 0.993
min normal dot expected radial ~= 0.990
status: PASS
```

Interpretation:

- The mesh normal direction and Neumann sign convention are consistent on the generated sphere.
- This supports the claim that the Neumann normal convention was validated before applying the solver to complex Bunny geometry.

Neumann normal convention figures:





7. Final Innovation Demo Results

The following results were generated after adding the final live-demo and self-diagnostic modes. They support the updated project framing:

A Self-Diagnostic and Optimization-Aware walk-on-Stars Solver for Complex Mesh Domains

Archived output folder:

results/formal_innovation_demo_20260603/

7.1 Walk Path Debugger / Live Demo

Command summary:

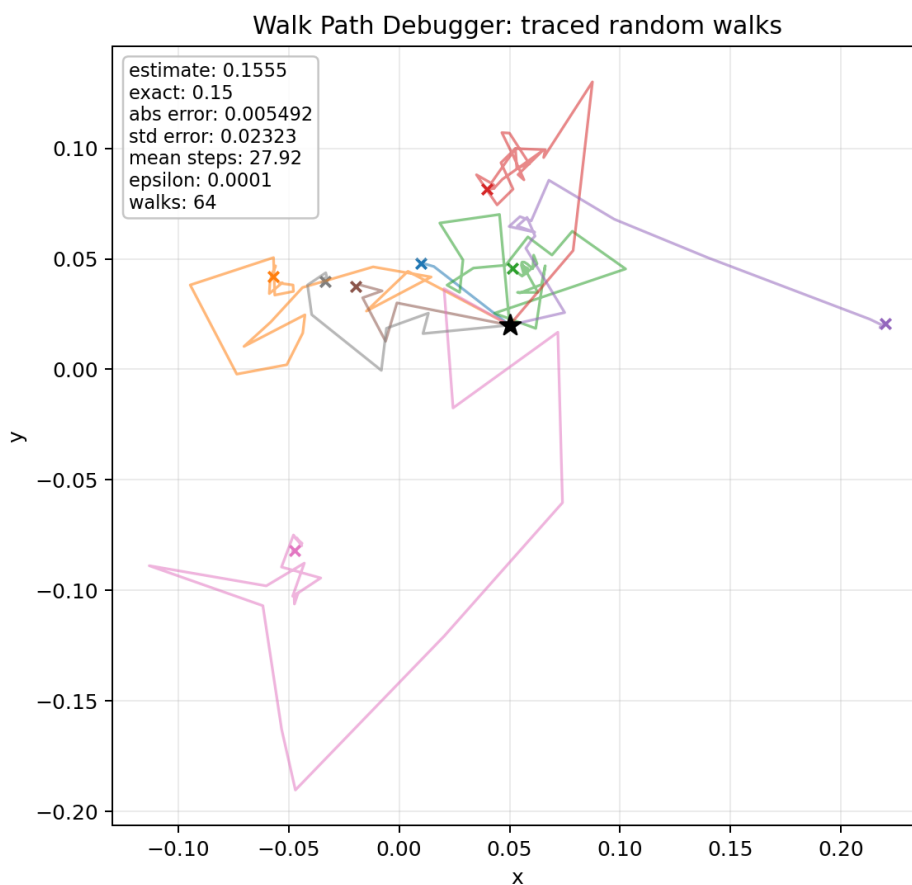
```
mode = demo_point
boundary = dirichlet
point = (0.05, 0.02, 0.08)
walks = 64
trace_walks = 8
epsilon = 1e-4
seed = 12345
```

Result:

Quantity	Value
Estimated value	0.155492
Exact value	0.150000
Absolute error	0.005492
Standard error	0.023233
Mean steps	27.921875
Runtime	0.0509 s
Samples used	64

Interpretation:

- The mode is fast enough for live presentation.
- The trace makes the random walk process visible rather than leaving WoSt as a black-box estimator.
- The summary links the visual trace to quantitative Monte Carlo diagnostics.



7.2 Boundary Bias Detector

The boundary-bias detector compares:

```
u_epsilon(x) and u_epsilon/2(x)
bias_indicator = |u_epsilon - u_epsilon/2|
normalized_bias = bias_indicator / (stdErr_epsilon + stdErr_epsilon_half + 1e-6)
```

Formal demo configuration:

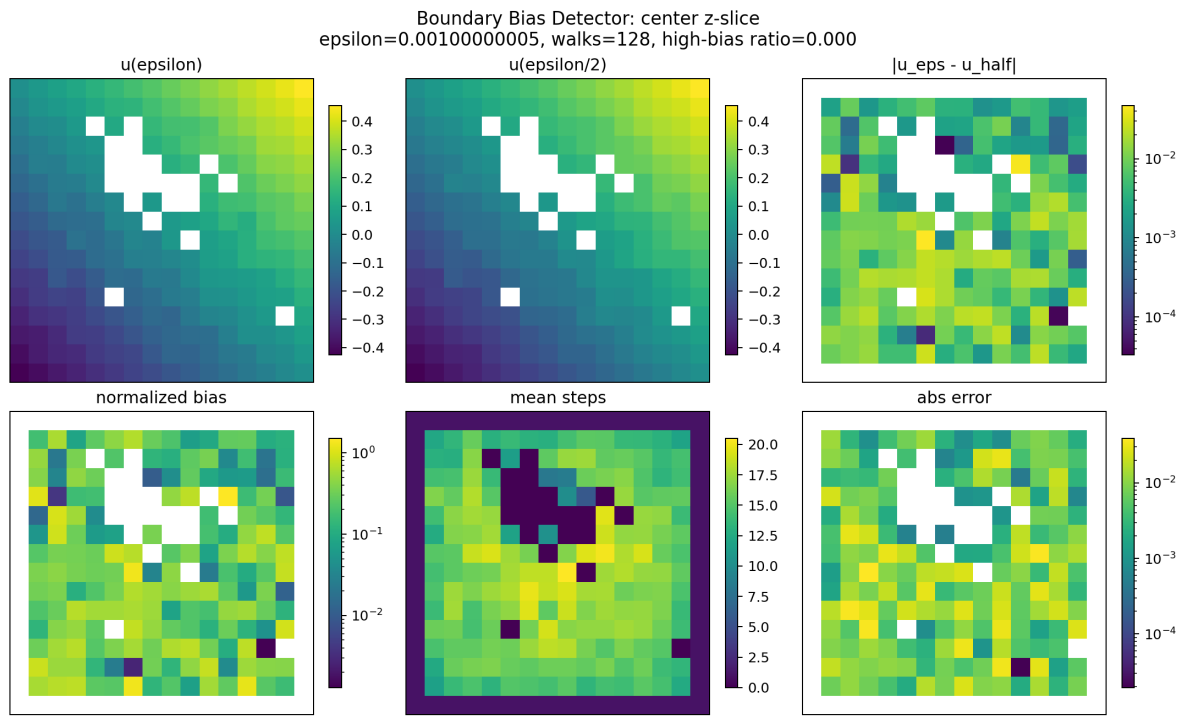
```
boundary = dirichlet
epsilon = 1e-3
epsilon_half = 5e-4
walks = 128
grid = 16^3
bias_threshold = 2.0
```

Result:

Quantity	Value
Valid points	4018 / 4096
Mean bias	0.006007
Max bias	0.064630
Mean normalized bias	0.246161
High-bias point count	0
High-bias ratio	0.000
RMSE at epsilon	0.010453
RMSE at epsilon/2	0.010230
Runtime	85.21 s

Interpretation:

- This run demonstrates the diagnostic mechanism on the Dirichlet Bunny grid.
- The low normalized-bias values indicate that, for this configuration, the epsilon sensitivity is mostly within Monte Carlo noise.
- The same detector is especially important for mixed Neumann cases, where the formal epsilon sweep showed large coarse-epsilon boundary bias.
- The requested `grid=32, walks=256` setting was too slow for this machine during live preparation, so the report uses the lighter `grid=16, walks=128` formal demo setting while preserving the same output fields and interpretation.



7.3 Variance-Predicted Adaptive Sampling

This mode estimates pilot variance with `m_pilot=32`, predicts the sample count using:

```
M_i = ceil(variance_i / tau^2)
M_i is clamped to [32, 1024]
```

Formal configuration:

```
queries = 500
valid points = 490
epsilon = 1e-4
pilot_samples = 32
min_samples = 32
max_samples = 1024
```

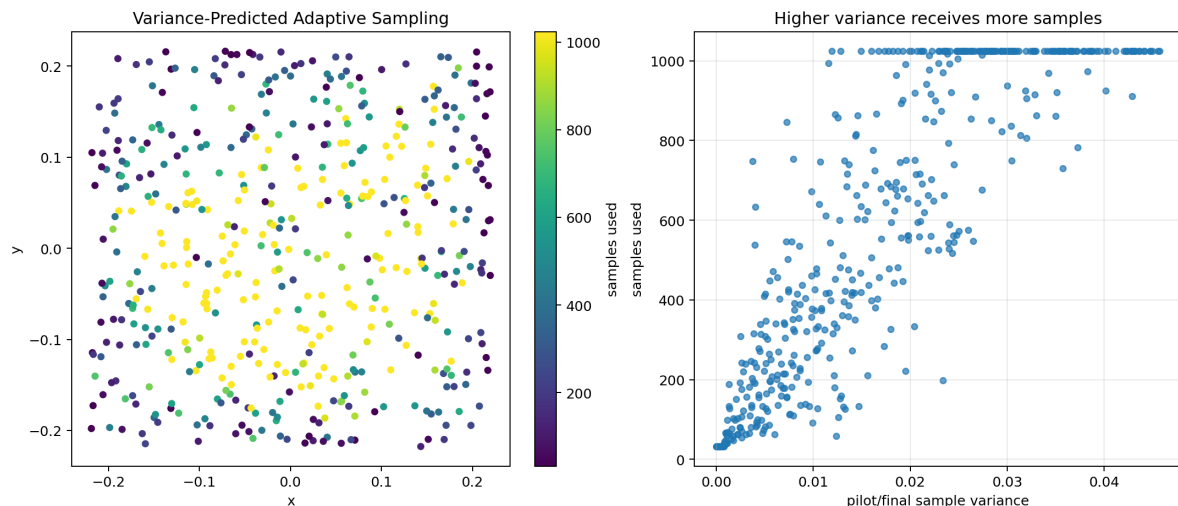
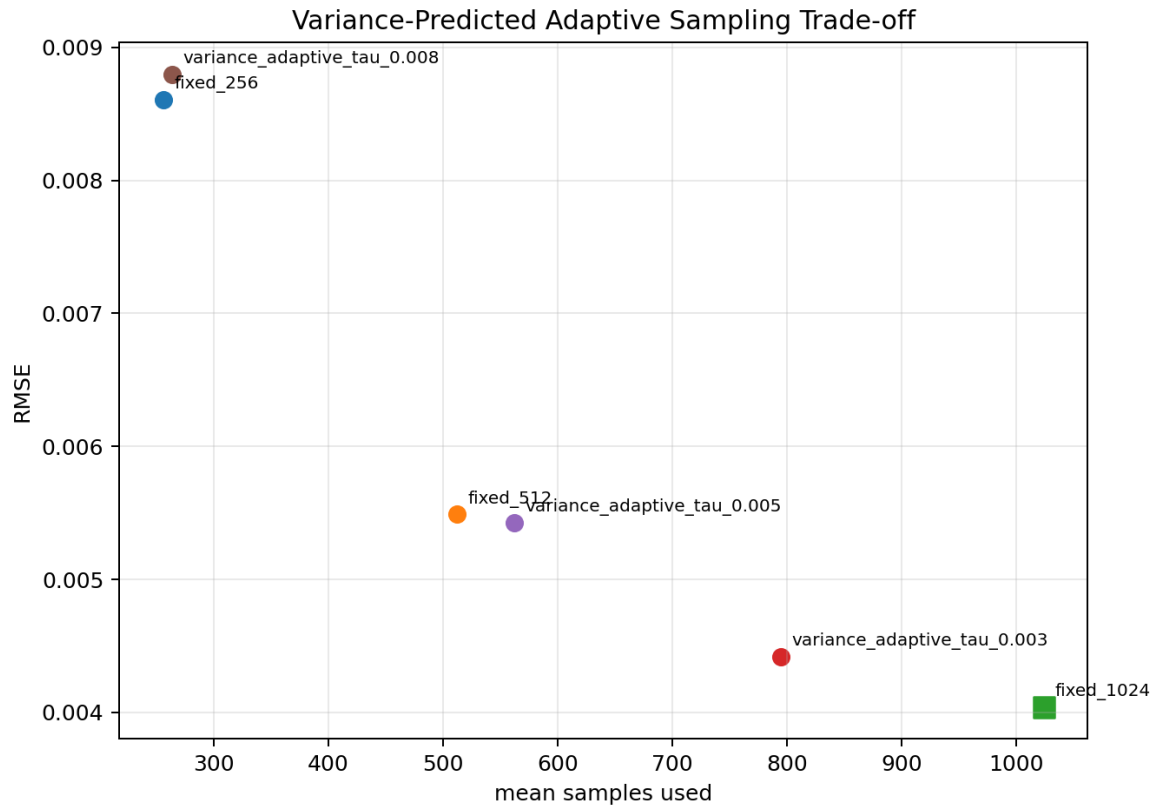
Comparison:

Method	RMSE	MAE	Mean samples	Runtime (s)
fixed_256	0.008607	0.006176	256.00	13.50
fixed_512	0.005489	0.004034	512.00	27.03
fixed_1024	0.004040	0.002918	1024.00	53.97
variance_adaptive_tau_0.003	0.004419	0.003335	794.80	45.81
variance_adaptive_tau_0.005	0.005426	0.004211	562.48	33.20
variance_adaptive_tau_0.008	0.008798	0.006482	263.78	16.58

Interpretation:

- `tau=0.005` is the most presentation-friendly trade-off: it reduces mean samples from `1024` to `562.48`, a reduction of about `45.1%`.

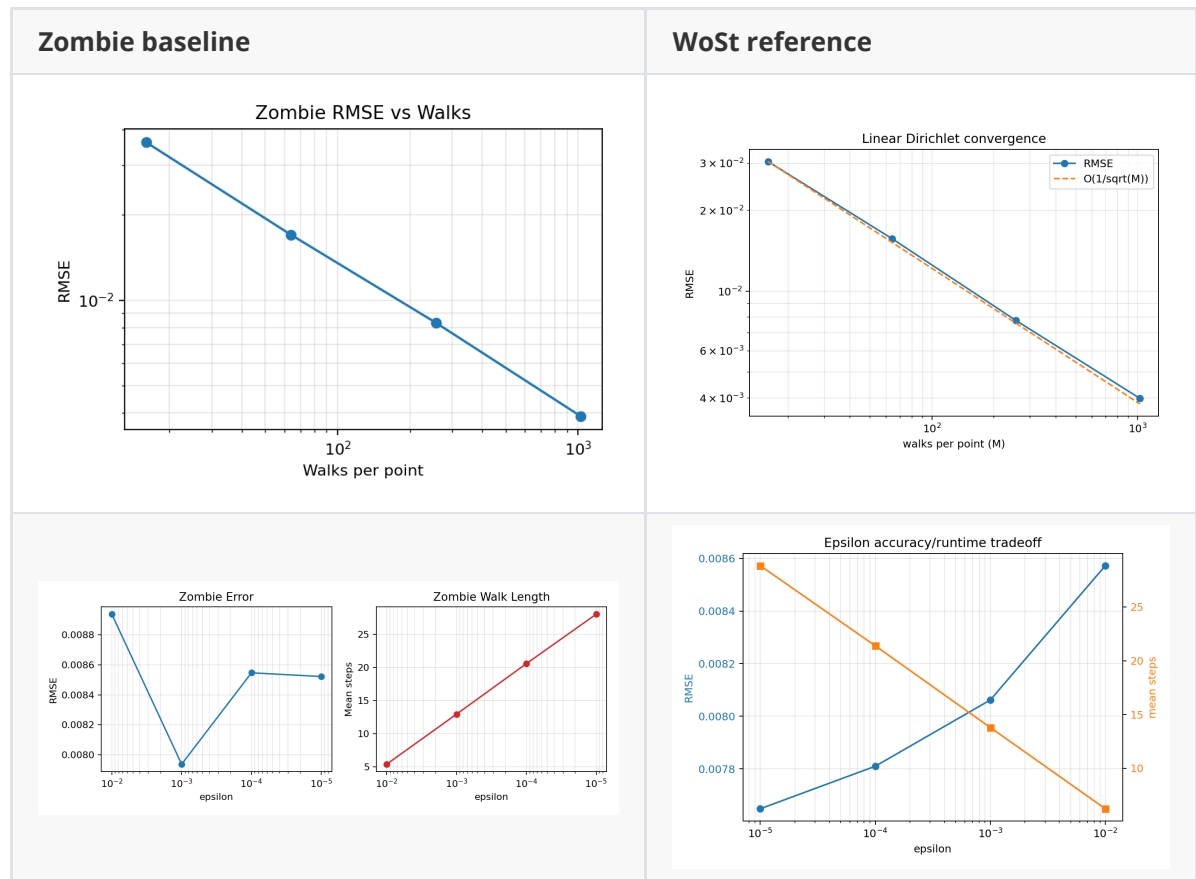
- Runtime drops from 53.97s to 33.20s, about 38.5% faster than fixed_1024.
- RMSE increases from 0.00404 to 0.00543, which is a visible but controlled accuracy-cost trade-off.
- $\tau=0.003$ stays close to fixed_1024 accuracy with fewer samples, while $\tau=0.008$ is faster but too noisy for high-accuracy reporting.
- The samples map and variance/sample scatter plot show the intended behavior: higher-variance points receive more predicted samples.



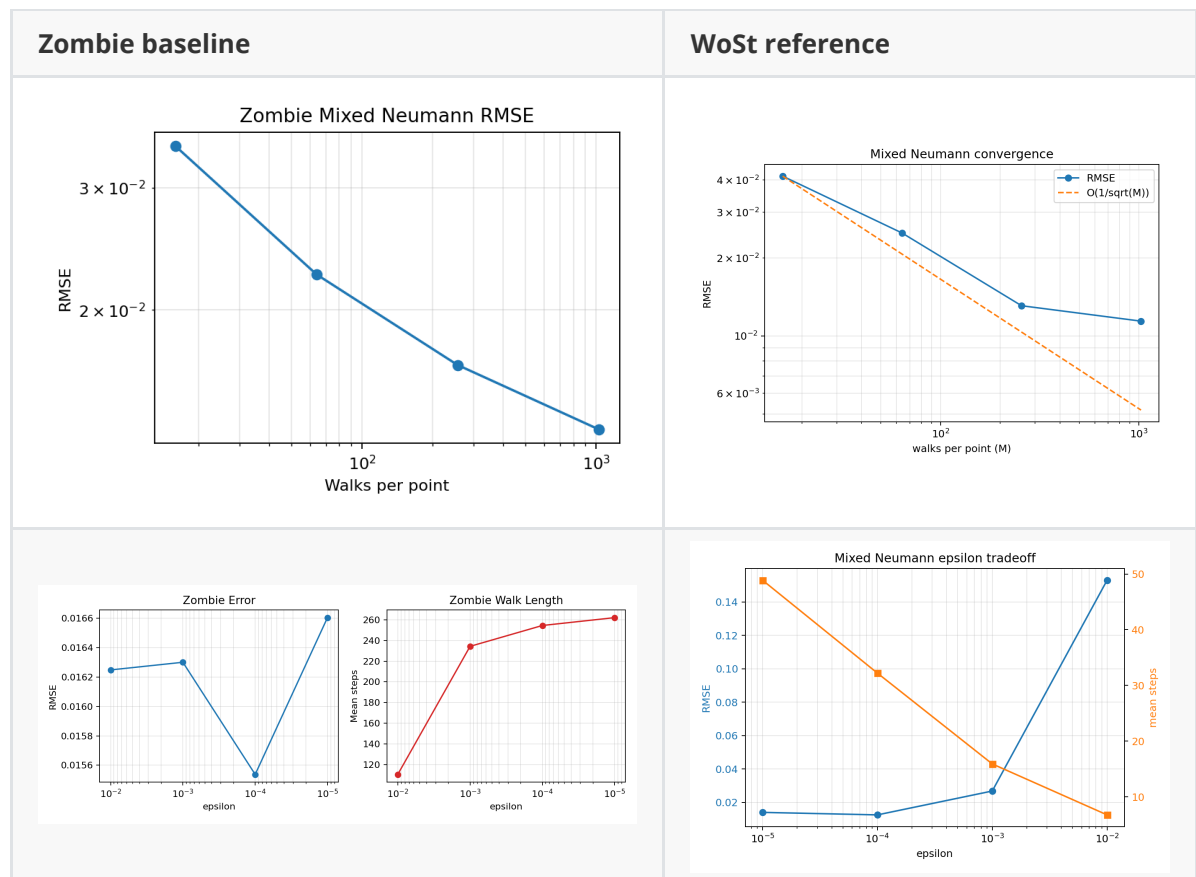
8. Historical Reference Figures from Zombie Final Report

The fresh formal comparison above is the primary quantitative source. The following figures are included because they were referenced in the earlier Zombie final report and are useful visual context.

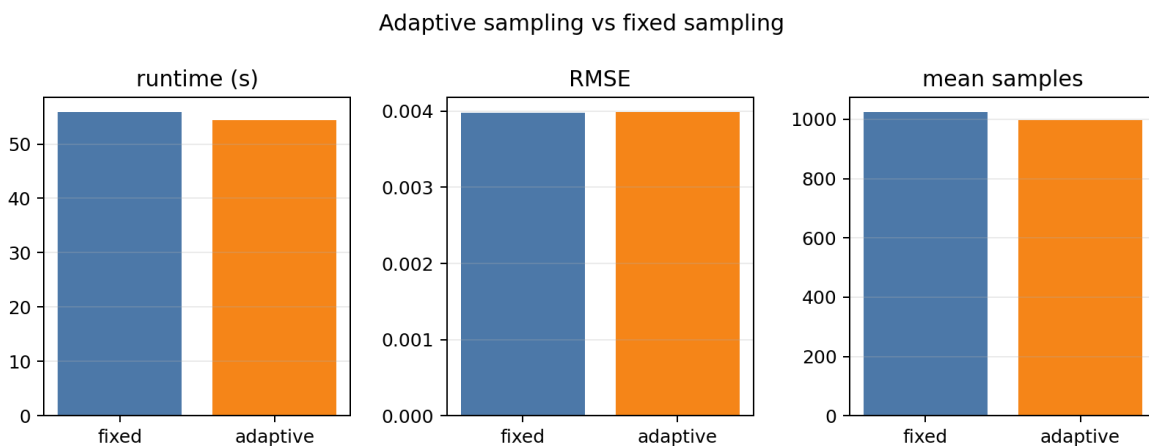
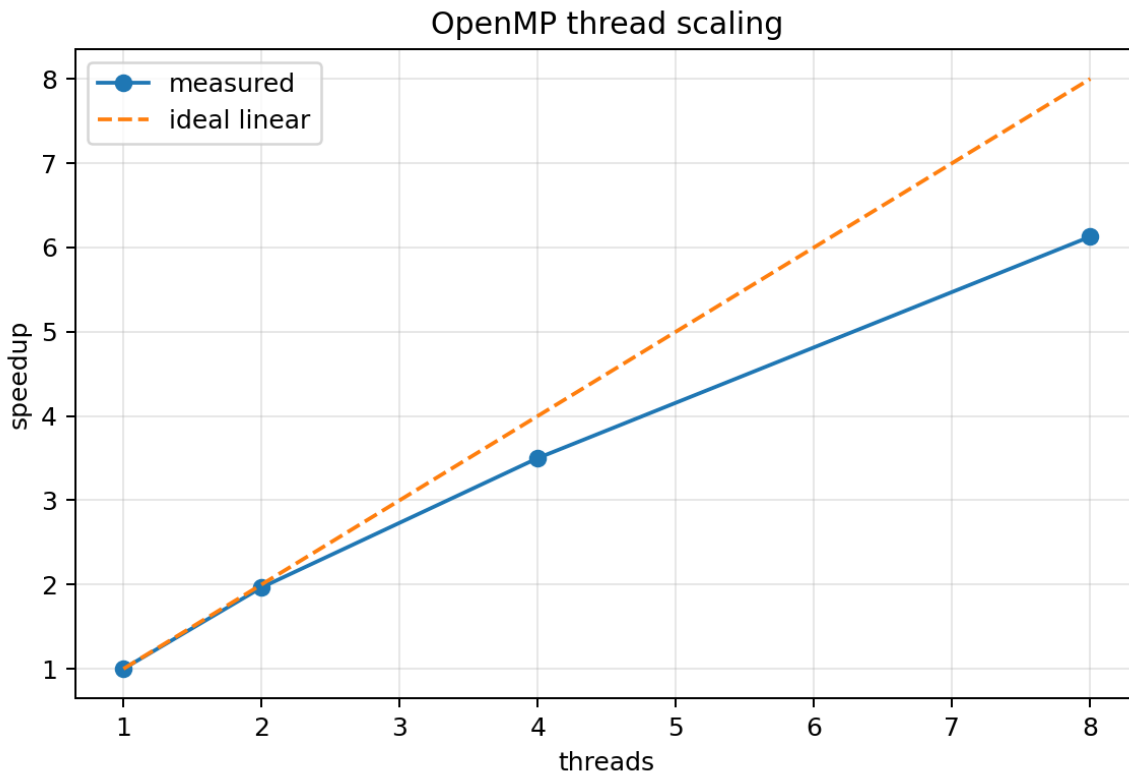
8.1 Historical Dirichlet Figures



8.2 Historical Neumann Figures



8.3 Historical WoSt-Only Figures



9. Limitations and Careful Claims

Safe claims:

- The Bunny benchmark uses a high-resolution 70,580-face mesh.
- WoSt and Zombie closely agree on the clean Dirichlet analytic benchmark.
- Both methods show Monte Carlo convergence with increasing random walks.
- Mixed Neumann behavior is more implementation-sensitive than Dirichlet behavior.
- WoSt's Neumann implementation is faster than Zombie at practical epsilon values in the fresh formal run.
- WoSt now includes meaningful optimization diagnostics for adaptive sampling, antithetic sampling, lazy refinement, CRN, epsilon sensitivity, and normal convention.
- WoSt now includes presentation-ready self-diagnostic modes: walk-path tracing, boundary-bias detection, and variance-predicted sample allocation.

Avoid overclaiming:

- Do not claim million-triangle scalability unless a million-triangle mesh is actually tested.

- Do not claim perfect linear speedup unless the thread sweep supports it for the target machine and workload.
- Do not treat the Zombie FCPW vs WoSt tiny_bvh timing as a pure backend-only comparison, because Zombie timing is measured through Python-loop calls.
- Do not claim that relative-standard-error adaptive sampling is fully tuned for Bunny-scale production until a full-size tuned run is completed.
- Do not claim the boundary-bias detector's `grid=32, walks=256` setting finished on this machine; the report uses the completed `grid=16, walks=128` formal demo run.

10. Final Summary

The final combined evidence supports the following project story:

wost is accurate on the clean Dirichlet Bunny benchmark and agrees closely with the Zombie baseline. The method also supports mixed Neumann boundaries, validated first with a sphere/cube normal-convention sanity test and then benchmarked on Bunny. Compared with Zombie, wost is slightly more accurate but slower in the Dirichlet solve-loop formal run; in the mixed Neumann formal run, wost is faster and competitive or better at practical epsilon values because it uses much shorter reflected paths. Additional wost optimization experiments show that relative-standard-error adaptive sampling can reduce sample counts, antithetic sampling can reduce variance, and lazy star-radius refinement can dramatically reduce geometric query cost without changing RMSE in the tested diagnostic configuration. The final innovation demo extends the project beyond reproduction: the solver can visualize walk paths, diagnose epsilon-sensitive boundary regions, and allocate samples using pilot variance predictions.

11. Key Files

Reports:

```
C:\THU\homework\zombie\results_zombie_final_report\ZOMBIE_VS_WOST_FINAL_REPORT.md
experiments/optimization_report.md
experiments/formal_comparison_report.md
experiments/final_integrated_report.md
```

Formal CSVs:

```
experiments/formal_wost_dirichlet/results/benchmark_summary.csv
experiments/formal_wost_dirichlet/results/geometry_benchmark.csv
experiments/formal_wost_neumann/results/benchmark_summary.csv
experiments/formal_zombie_dirichlet/benchmark_summary.csv
experiments/formal_zombie_dirichlet/zombie_vs_wost_summary.csv
experiments/formal_zombie_neumann/benchmark_summary.csv
experiments/formal_zombie_neumann/zombie_vs_wost_neumann_summary.csv
```

Optimization figures:

```
experiments/figures/adaptive_rmse_errorbars.png
experiments/figures/adaptive_samples_errorbars.png
experiments/figures/antithetic_pointwise_variance.png
experiments/figures/antithetic_rmse_errorbars.png
experiments/figures/antithetic_variance_errorbars.png
experiments/figures/lazy_refinement_ratio_errorbars.png
experiments/figures/lazy_tradeoff_rmse_vs_refinement.png
experiments/figures/epsilon_sensitivity_histogram.png
experiments/figures/neumann_normal_diagnostics.png
```

Final innovation demo outputs:

```
results/formal_innovation_demo_20260603/live_trace.csv
results/formal_innovation_demo_20260603/live_demo_summary.csv
results/formal_innovation_demo_20260603/live_trace_plot.png
results/formal_innovation_demo_20260603/boundary_bias_detector.vtk
results/formal_innovation_demo_20260603/boundary_bias_summary.csv
results/formal_innovation_demo_20260603/boundary_bias_detector.png
results/formal_innovation_demo_20260603/variance_adaptive_points.csv
results/formal_innovation_demo_20260603/variance_adaptive_summary.csv
results/formal_innovation_demo_20260603/variance_adaptive_comparison.csv
results/formal_innovation_demo_20260603/variance_adaptive_tradeoff.png
results/formal_innovation_demo_20260603/variance_adaptive_samples_map.png
```

Final innovation demo figures included in this report:

```
experiments/final_report_figures/innovation_live_trace_plot.png
experiments/final_report_figures/innovation_boundary_bias_detector.png
experiments/final_report_figures/innovation_variance_adaptive_tradeoff.png
experiments/final_report_figures/innovation_variance_adaptive_samples_map.png
```

Integrated local figure folder:

```
experiments/final_report_figures/
```